Prerequisite Software

Our upcoming workshop/tutorial, **Cognitive Memory Architectures: Designing Continuity into Long-Running RAG and Agentic Solutions**, is hands-on and designed to give participants real-world experience building Agentic AI using components including vector embeddings, MCP to query real-time data, and Agent Memory, the star of this workshop.

To make the most of this interactive workshop, participants should ensure they have:

- A Linux or Mac-based developer laptop with enough memory (8GB+ minimum, 16GB recommended) to run a few podman/Docker containers.
    - Windows Users should use a VM or Cloud Instance. This will NOT be provided.
- Python installed: **3.12 is recommended (the only version I tested with and can guarantee will work)**
    - (**Optional, but HIGHLY Recommended**) Using a **miniconda (recommended for beginners)**, uv, or venv virtual environment
    - Familiarity with Python is a must.
- **Podman installed (recommended, as we will be using this)** or Docker Desktop
    - Podman installation instructions: https://podman.io/docs/installation
    - Pull container images BEFORE the day of the workshop:
        - Script:
            - Visit: https://github.com/davidvonthenen/2026-odsc-west
        - Manual method: pre-pull the following Podman images:
            - podman pull docker.io/opensearchproject/opensearch:3.5.0
            - podman pull docker.io/opensearchproject/opensearch-dashboards:3.5.0
            - podman pull pgvector/pgvector:pg17
            - Build mem0 and mem0-dashboard using these instructions: https://github.com/mem0ai/mem0/blob/main/server/README.md
- Access to an OpenAI-compatible platform (REST API) with an API KEY
    - **An API Key will NOT be provided**
    - Compatible platforms: OpenAI, Nebius TokenFactory, Groq
        - Untested but should work: Together AI, Fireworks AI, or OpenRouter
    - If you don't have access to an API Key, you can use a local Small Language Model, but that requires downloading these models and dropping them in `~/models` on your laptop. (Requires a recommended 16GB of memory)
        - Apple laptops: https://huggingface.co/mlx-community/Qwen2.5-7B-Instruct-1M-4bit
        - All other CPUs: https://huggingface.co/bartowski/Qwen2.5-7B-Instruct-1M-GGUF/blob/main/Qwen2.5-7B-Instruct-1M-Q5_K_M.gguf

This workshop is geared toward the real world, where AI solutions are deployed on hardware (aka your laptop). We likely won't use Jupyter Notebooks for this workshop, since it builds up to an Agentic RAG agent using a few Podman containers, which is difficult to pull off in Jupyter Notebooks.

If you don't have the hardware to do the hands-on portion, there will be a presentation at the beginning that you can listen to and participate in. Towards the end of the workshop, I will go through each piece of the hands-on part as the final demo.

The workshop will start with a simple RAG Agent and progressively build toward a stateful Agentic RAG solution with durable/persistent memory, focusing on memory-based operations.

Stages:

- Simple RAG
- RAG + MCP (real-time data) with Memory
- Agentic RAG + MCP (real-time data) with Memory
- Explicit Memory Lifecycle Policies: Long-running systems that consolidate, retrieve, update, and forget memories under explicit policies.

Attendees will:

- Understand episodic and semantic memory and persistent Agentic Memory.
- Consolidate episodic records into compact semantic memories while preserving provenance.
- Resolve conflicting, outdated, or superseded memories.
- Apply controlled forgetting through expiration and deletion.
- Use deterministic harness code to scope, validate, retrieve, and store structured memory.
- Understand how AI governance and Agent Memory are important considerations when building Agentic solutions.